



Faculdade de Ciências da Universidade do Porto

Licenciatura de Inteligência Artificial e Ciência de Dados

Evaluating Logarithmic Number Systems for Low-Resource Language Models

github.com/specs-feup/LNS-Library

Autor

Henrique dos Santos Teixeira

Junho de 2026

Resumo

O Logarithmic Number System (LNS) representa os valores como logaritmos de vírgula fixa em complemento para dois, permitindo que a multiplicação e a divisão sejam realizadas através de simples adições e subtrações inteiras e que a raiz quadrada se reduza a um deslocamento aritmético à direita. Esta propriedade elimina os multiplicadores de mantissa do hardware, reduzindo significativamente a área e o consumo de energia, características especialmente relevantes para a inferência de redes neuronais em dispositivos de baixos recursos.

Este estágio, realizado no INESC TEC, teve quatro objectivos principais: em primeiro lugar, acrescentar a trabalho anterior relativo a LNS, onde operações aritméticas foram implementadas, a capacidade para operações de comparação de números LNS, assim como operações de conversão entre Floating Point/Brain Float (FP/BF) e LNS; em segundo lugar, criar uma biblioteca de emulação LNS que permitisse explorar o desempenho do formato em função de vários parâmetros; em terceiro lugar, realizar uma comparação estatística entre LNS e BF da precisão aritmética, em múltiplos intervalos, assim como testes em kernels; em quarto lugar, avaliar a performance do formato via um caso de teste relevante, nomeadamente, inferência em LLMs da arquitectura Llama 2.

No regime de 8 bits, o **bf8** é superior em termos de erro absoluto e relativo nas operações aritméticas, no entanto, o **lns8** tem apenas uma pequena desvantagem sob o **bf8** na multiplicação e divisão em todos os intervalos testados, enquanto tem uma maior vantagem na adição, subtracção e round-trip (conversão de um número em **fp32** para o formato e conversão de volta para **fp32**). Isto reflecte directamente a natureza do LNS, onde a multiplicação e a divisão são operações exactas no domínio logarítmico, ao passo que a adição e a subtracção requerem funções de correção não-lineares sujeitas a erro de aproximação. No regime de 16 bits, o **bf16** domina o erro relativo na multiplicação, adição e subtracção, enquanto o **lns16** mantém vantagem consistente na divisão em todos os intervalos e no erro absoluto da multiplicação. Nos testes em kernels, ambos os formatos de 8 bits têm um erro relativo equivalente abaixo de 1.0 em tarefas de acumulação intensiva como uma progressão geométrica e uma série harmónica alternada, evidenciando as limitações inerentes à precisão reduzida, enquanto que em 16 bits, o LNS apresenta um erro significativamente inferior ao BF. Também no regime de 16 bits, o **lns16** apresenta erro superior ao **bf16** em funções como a GELU, cuja avaliação envolve composição de operações não-lineares pouco favoráveis ao domínio logarítmico, no entanto, o **lns16** apresenta um menor erro na Softmax face ao **bf16**. A inferência do modelo *stories15M* e *stories42M* com pesos convertidos directamente de **fp32** para **lns16** produz texto coerente, confirmando a viabilidade prática do formato para tarefas de inferência de maior complexidade.

Agradecimentos

Agradeço ao meu supervisor Prof. Nuno Paulino pela orientação constante e pelo rigor com que acompanhou este trabalho, bem como aos co-supervisores Luís Sousa e Guilherme Oliveira pelo apoio técnico e pelas discussões produtivas ao longo de todo o estágio. Agradeço também ao INESC TEC pelo ambiente de investigação e pelos recursos disponibilizados.

Índice

1	Introdução	2
2	Estado da Arte e Enquadramento	3
2.1	Estado da Arte	3
2.2	Enquadramento	6
3	Descrição do Trabalho	8
3.1	Comparações em LNS e Conversão FP/BF $\leftrightarrow$ LNS	8
3.2	Biblioteca de Emulação LNS	9
3.3	Comparação de Precisão Aritmética: LNS vs. BF	10
3.4	Inferência na Arquitectura Llama 2	11
4	Resultados e Análise	12
4.1	Comparação de Precisão Aritmética: LNS vs. BF	12
4.2	Inferência na Arquitetura Llama 2	15
5	Conclusão e Trabalho Futuro	17
	Bibliografia	18
A	Anexo	19
A.1	APIs	19
A.2	Spline	22

Lista de Figuras

Figura 1	Pseudocódigo do Algoritmo Greedy para Tabelas Spline XF e XMB	7
Figura 2	Erro relativo e absoluto médio por intervalo de valores para os operadores. (p é o número de bits da parte inteira do expoente em LNS — 2^{-p+1} é o número mais pequeno que ambos os formatos conseguem representar)	12
Figura 3	Comparação de heatmaps de vencedor por operação $\times$ intervalo para os formatos de 8 bits (<code>lns8</code> vs <code>bf8</code>). À esquerda: erro relativo médio; À direita: erro absoluto médio.	13
Figura 4	Erro relativo médio dos testes numéricos para todos os formatos. Os empates são anotados quando a diferença relativa entre <code>lns</code> e <code>bf</code> é inferior a 5%.	14
Figura 5	Erro absoluto médio dos testes numéricos para todos os formatos. Os empates são anotados quando a diferença relativa entre <code>lns</code> e <code>bf</code> é inferior a 5%.	14

Lista de Tabelas

Tabela 1	Especificações dos formatos LNS suportados.	6
Tabela 2	Parâmetros de inferência utilizados nos modelos Llama 2.	15

Abreviaturas e Siglas

BF	Brain Float
CPU	Central Processing Unit
DSP	Digital Signal Processing block
FP	Floating Point
FPGA	Field-Programmable Gate Array
FPU	Floating Point Unit
HLS	High-Level Synthesis
LLM	Large Language Model
LNS	Logarithmic Number System
LNSU	Logarithmic Number System Unit
LUT	Look-Up Table
MAE	Mean Absolute Error
MRE	Mean Relative Error
RISC	Reduced Instruction Set Computer
XF	Formato de armazenamento abcissa-ordenada (x, f) dos Splines
XMB	Formato de armazenamento abcissa-declive-ordenada-na-origem (x, m, b) dos Splines

1 Introdução

A crescente utilização de redes neuronais em dispositivos de baixos recursos tem intensificado a necessidade de plataformas de computação energeticamente eficientes. Embora o treino de modelos seja frequentemente realizado com aritmética de Floating Point (FP) de 32 bits (fp32), a fase de inferência é significativamente mais tolerante à perda de precisão. Como a aritmética IEEE 754 é dispendiosa em termos de hardware e consumo de energia, existe um esforço crescente de investigação orientado para representações numéricas alternativas, incluindo Brain Float (BF), Low-Precision Floating Point, vírgula fixa, posits e outros formatos especializados.

O **Logarithmic Number System** (LNS) constitui uma alternativa particularmente atraente. Ao representar os valores como logaritmos de vírgula fixa em complemento para dois, o LNS permite utilizar apenas somas, subtrações e deslocamentos de bits para implementação de multiplicações, divisões e exponenciações com potências de dois (como raízes quadradas, quartas, ... ou elevar a dois, quatro, ...) simplificando drasticamente a complexidade do hardware. Estas propriedades eliminam os multiplicadores de mantissa do hardware, o que pode reduzir substancialmente a área e o consumo de energia. Adicionalmente, o LNS oferece maior densidade de representação na gama $[-1, 1]$, onde se concentram os pesos (parâmetros) típicos de redes neuronais treinadas.

O trabalho realizado neste estágio insere-se num projecto mais amplo desenvolvido no INESC TEC, cujo objectivo é a continuação do desenvolvimento de uma extensão LNS num CPU RISC-V (RISC++), implementado através de High Level Synthesis (HLS), um processo de design automatizado que converte software abstrato (como C, C++ ou SystemC) em circuitos digitais de hardware (Register-Transfer Level — RTL). Na implementação existente para suporte de LNS neste processador [1], existia já o suporte para os dois formatos LNS `lns16 Q8.7` e `lns8 Q4.3`, assim como uma LNSU com suporte para as operações `add`, `sub`, `mul`, `div` e `sqrt`. A LNSU recorre a uma aproximação por splines para as funções não-lineares intrínsecas à adição e subtração gerados por um algoritmo greedy, obtendo precisão de 2^{-7} e 2^{-3} , respetivamente, com tabelas de no máximo 324 bytes. O presente estágio teve por objectivo continuar e aprofundar esse trabalho em quatro direcções principais:

1. **adicionar operações de comparação e conversão** de FP/BF $\leftrightarrow$ LNS;
2. **criar uma biblioteca de emulação LNS** para criar uma base de exploração do formato;
3. **comparar o LNS ao BF** em precisão aritmética com testes estatísticos;
4. **suportar inferência em LLMs**, nomeadamente a arquitectura Llama 2.

Este relatório está estruturado da seguinte forma:

- a Secção 2 apresenta o estado da arte e o enquadramento;
- a Secção 3 descreve em detalhe o trabalho realizado;
- a Secção 4 reúne os resultados e procede à análise destes;
- a Secção 5 expõe as conclusões e perspectivas de trabalho futuro.

2 Estado da Arte e Enquadramento

2.1 Estado da Arte

2.1.1 O Logarithmic Number System

Parhami [2] constitui a referência canónica do LNS, descrevendo-o como uma alternativa estruturalmente distinta à vírgula flutuante: ao representar os valores como logaritmos de vírgula fixa em complemento para dois, o sistema transforma a multiplicação em adição inteira e a divisão em subtracção inteira.

Em LNS, um número real não nulo x é representado por um bit de sinal S_x e um logaritmo binário de vírgula fixa L_x em complemento para dois:

$$S_x = \mathbb{1}[x < 0] \quad L_x = \log_2(|x|) \quad x = (-1)^{S_x} \cdot 2^{L_x}$$

O expoente L_x é um número de vírgula fixa em complemento para dois, com uma parte inteira e uma parte fraccionária. Valores negativos de L_x representam magnitudes em $(0, 1)$ e valores positivos representam magnitudes em $(1, +\infty)$, o que confere ao LNS maior densidade de representação na gama $[-1, 1]$ — precisamente onde se concentram os pesos típicos de redes neuronais treinadas.

Segue-se a tabela com a equivalência das operações em LNS:

z	S_z	L_z	Condição
$x + y$	S_x	$L_x + \log_2(1 + 2^{L_y - L_x})$	$\forall x, y, x > y $
$x - y$	S_x	$L_x + \log_2(1 - 2^{L_y - L_x})$	$\forall x, y, x > y $
$x \cdot y$	$S_x \oplus S_y$	$L_x + L_y$	$\forall x, y$
$x \div y$	$S_x \oplus S_y$	$L_x - L_y$	$\forall x, y$
x^{2^n}	0	$L_x \ll n$	$\forall n > 1$
$x^{2^{-n}}$	0	$L_x \gg n$	$\forall n > 1, x \geq 0$

A multiplicação reduz-se a uma adição inteira dos expoentes, a divisão a uma subtração e as potências ou raízes de base 2 a meros deslocamentos lógicos à esquerda ou à direita (shifts) — todos significativamente mais simples e eficientes em termos energéticos do que as suas contrapartes em vírgula flutuante, que requerem manipulação explícita de mantissas.

A adição e a subtracção requerem cuidado adicional. Para $z = x \pm y$ com $|x| \geq |y|$, a derivação parte de:

$$\begin{aligned}
 z = x \pm y &\Rightarrow L_z = \log_2(|x \pm y|) \\
 &= \log_2(|2^{L_x} \pm 2^{L_y}|) \\
 &= \log_2(2^{L_x} \cdot |1 \pm 2^{L_y - L_x}|) \\
 &= L_x + \log_2(|1 \pm 2^{L_y - L_x}|) \\
 &= L_x + \log_2(1 \pm 2^{L_y - L_x})
 \end{aligned}$$

Definindo $t = L_y - L_x \leq 0$ (garantido por troca de operandos se necessário), introduz-se a chamada **função de Gauss logarítmica de adição**:

$$f^+(t) = \log_2(1 + 2^t), \quad t \in (-\infty, 0]$$

e, analogamente para a subtração, a **função de Gauss logarítmica de subtração**:

$$f^-(t) = \log_2(1 - 2^t), \quad t \in (-\infty, 0)$$

A função f^+ é suave e limitada em $(-\infty, 0]$. Cresce monotonicamente de 0 quando $t \rightarrow -\infty$ até 1 em $t = 0$. As suas primeira e segunda derivadas são:

$$\frac{d}{dt}f^+(t) = \frac{2^t}{1 + 2^t}, \quad \frac{d^2}{dt^2}f^+(t) = \ln(2) \cdot \frac{2^t}{(1 + 2^t)^2}$$

Ambas são sempre positivas, monotonicamente crescentes, e aproximam-se de 0 quando $t \rightarrow -\infty$ e dos valores $\frac{1}{2}$ e $\frac{\ln(2)}{4}$ em $t = 0$. A suavidade de f^+ torna-a favorável à aproximação, visto que poucas peças são suficientes para atingir alta precisão.

A função f^- está definida apenas em $(-\infty, 0)$. Quando $t \rightarrow 0^-$, tem-se $f^-(t) \rightarrow -\infty$, devido à singularidade do logaritmo em zero. As suas derivadas:

$$\frac{d}{dt}f^-(t) = -\frac{2^t}{1 - 2^t}, \quad \frac{d^2}{dt^2}f^-(t) = -\ln(2) \cdot \frac{2^t}{(1 - 2^t)^2}$$

são sempre negativas e divergem para $-\infty$ quando $t \rightarrow 0^-$.

Em hardware, f^+ e f^- são tipicamente aproximadas por splines lineares, normalmente via tabelas de consulta (LUTs), dada a sua simplicidade de implementação. Coleman *et al.* [3] revêem diversas estratégias para a avaliação eficiente destas funções, central para a implementação prática do LNS. A segunda derivada não limitada perto de $t = 0$ é a razão fundamental pela qual a função de subtração requer significativamente mais peças do que a função de adição para atingir a mesma precisão, uma limitação que decorre da curvatura da própria função, não do método de aproximação escolhido.

As derivações detalhadas encontram-se em `docs/docs.pdf` no repositório do projecto*.

*<https://github.com/specs-feup/LNS-Library/blob/main/docs/docs.pdf>

2.1.2 LNS em Aceleração de Redes Neurais

Christ et al. [4] apresentam uma unidade LNS de baixa precisão para aceleradores de redes neurais em FPGA Kintex-7, obtendo degradação de apenas 3% na precisão top-1 no MNIST com 12 500 a 14 000 LUTs. A arquitectura partilha o objectivo de minimizar o custo das funções de correção via LUTs de dimensão reduzida. A diferença central é de escala: enquanto Christ et al. [4] focam redes convolucionais compactas, o presente trabalho estende a análise para inferência em modelos Llama 2 (stories15M e stories42M), com cargas de trabalho substancialmente mais intensas.

2.1.3 Formatos Alternativos: Posits e Microscaling

Wu et al. [5] apresentam o PPU, uma extensão RISC-V para aritmética posit vectorial. Os posits oferecem maior precisão efectiva por bit em certas gamas, mas a custos de hardware consideráveis: o PERCIVAL requer $2.94\times$ mais LUTs e $3.0\times$ mais flip-flops do que uma FPU convencional. Em contraste, a LNSU do trabalho anterior [1] requer até 45% menos LUTs, 48% menos flip-flops e 58% menos DSPs, com frequências 29%–36% superiores. Os formatos de microscaling (MX) [6], [7] adoptam abordagem diferente: introduzem um expoente partilhado por blocos de elementos, reduzindo o custo de representação sem alterar a aritmética. O MXDOTP [7] obtém acelerações de até $3.4\times$ face ao `fp32` em multiplicações de matrizes. Estes formatos são ortogonais ao LNS e poderiam, em princípio, ser combinados.

2.1.4 Compressão e Quantização de LLMs

Han et al. [8] demonstraram que redes neurais profundas toleram quantização agressiva dos pesos sem degradação significativa, desde que a estrutura do modelo seja preservada — motivação directa para a conversão directa `fp32` $\rightarrow$ `lns16` sem re-treino adoptada neste trabalho. A arquitectura Llama 2 nos modelos *TinyStories* [9] utiliza multi-head attention e normalização RMS, operações cujo comportamento numérico em formatos de baixa precisão é menos estudado do que o das redes convolucionais clássicas. Neste trabalho, a produção de texto coerente após conversão directa confirma que o `lns16 Q8.7` é uma alternativa prática ao `bf16` para inferência em dispositivos de recursos limitados.

2.2 Enquadramento

A assimetria mencionada na Secção 2.1 sobre a dificuldade de aproximação entre a função de Gauss logarítmica de adição e subtração é confirmada analiticamente e documentada por Coleman *et al.* [3], que demonstram que f^- requer significativamente mais entradas de LUT do que f^+ devido à singularidade em $t = 0$, explorando ainda compromissos entre dimensão das tabelas e precisão, o que motiva directamente a exploração das seguintes representações das tabelas de spline.

2.2.1 Representações XF e XMB

O ponto de partida da implementação deste relatório foi um trabalho anterior onde as funções de soma e subtração em LNS foram implementadas com recurso a dois métodos de aproximação. Nomeadamente, foram aproximadas as funções f^+ e f^- via construção de splines gerados por um algoritmo greedy, onde os parametros necessários para avaliação dos splines em tempo de execução tomou o formato de XF, onde são armazenados os pares abcissa-ordenada, ou XMB, onde são armazenados os pares abcissa-declive-ordenada-na-origem.

Dado um conjunto de $n + 1$ pontos $\{(x_k, f_k)\}_{k=0}^{\{n\}}$, a fórmula da interpolação de um spline linear para um intervalo $[x_{k-1}, x_k]$ é:

$$LS_k(x) = \frac{(x_k - x)f_{k-1} + (x - x_{k-1})f_k}{h_k}, \quad h_k = x_k - x_{k-1}$$

Isto é a representação XF. Uma forma equivalente é:

$$LS_k(x) = m_k \cdot x + b_k, \quad m_k = \frac{f_k - f_{k-1}}{h_k}, \quad b_k = \frac{f_{k-1}x_k - f_kx_{k-1}}{h_k}$$

Esta é a representação XMB. Precomputa o declive m_k e a interceção b_k por intervalo, reduzindo a avaliação em runtime a um simples multiply-add. O trade-off é o aumento no uso da memória: XF requiere $4 \cdot (n + 1)$ bytes para uma implementação de `lns16`, enquanto o XMB requiere $6 \cdot (n + 1)$ bytes.

A fórmula para o upper bound do error de um spline linear é:

$$\varepsilon \leq \frac{1}{8} \max_{[a,b]} |f''(x)| h_i^2$$

2.2.2 Formatos Suportados

Não existe nenhuma especificação LNS, sendo que podem ser alocados para a mantissa e expoente quaisquer número de bits. No trabalho anterior e para a implementação e testes no presente trabalho, foram considerados dois formatos de LNS: um formato de 16 bits com configuração Q4.3 e um formato de 16 bits com configuração Q8.7, ilustrados na Tabela 1.

Formato	S_x (Sinal)	L_{x_i} (Inteira)	L_{x_f} (Fracionária)	Intervalo
<code>lns8 Q4.3</code>	1	4	3	2^{-8} a $2^{7.875}$
<code>lns16 Q8.7</code>	1	8	7	2^{-128} a $2^{127.992}$

Tabela 1: Especificações dos formatos LNS suportados.

O formato `lns16 Q8.7` tem 7 bits de mantissa efectiva, o mesmo que o `bf16`, tornando a comparação directa entre os dois formatos particularmente relevante.

2.2.3 Erros dos Splines de Aproximação

A fórmula dos erros para o spline de aproximação das funções f^+ e f^- são:

$$\varepsilon_k^+ \leq \frac{\ln(2) \cdot 2^{x_k} \cdot h_k^2}{8(1 + 2^{x_k})^2} \quad \varepsilon_k^- \leq \frac{\ln(2) \cdot 2^{x_k} \cdot h_k^2}{8(1 - 2^{x_k})^2}$$

Como a upper bound é monótona e crescente para ambas as funções, o intervalo a separar em dois é sempre aquele com maior h_k ponderado pela curvatura local. O mínimo tamanho de intervalo permitido é $2 \cdot 2^{-f}$, onde f é a parte fracionária do formato LNS.

2.2.4 Algoritmo Greedy

O algoritmo *greedy* começa com as duas extremidades do intervalo inicial $[-8, 0)$. Iterativamente adiciona o ponto intermédio do intervalo com maior estimativa de erro. Tanto para `lns8 Q4.3` como para `lns16 Q8.7`, $f^\pm(-8)$ é um número que atua como lower bound da representação de ambos os formatos.

Algorithm GreedySplineSelection($X, Y, M, B, S, f, \varepsilon$)

Inputs: Empty sequence X , Empty sequence Y , Empty sequence M , Empty sequence B , target size S , target function f , error metric ε

Output: Optimized spline table T

```

1.  $X \leftarrow [\text{start}, \text{end}]$ 
2. while  $|X| < S$  do
3.   for  $j \leftarrow 1$  to  $|X| - 1$  do
4.      $h_j \leftarrow X_{j+1} - X_j$ 
5.      $e_j \leftarrow \varepsilon(X_j, h_j)$ 
6.   end for
7.    $\text{max\_j} \leftarrow \arg \max_j (e_j)$  where  $h_j > 2 \cdot 2^{-f}$ 
8.    $X_{\text{new}} \leftarrow X_{\text{max\_j}} + (h_{\text{max\_j}} \gg 1)$ 
9.    $X_{\text{max\_j}+1} \leftarrow x_{\text{new}}$ 
10. end while

11. for  $i \leftarrow 1$  to  $|X|$  do
12.    $Y_i \leftarrow f(X_i)$   $\triangleright$  valor da ordenada y do ponto do formato XF
13. end for

14. for  $i \leftarrow 2$  to  $|X|$  do
15.    $k \leftarrow \lfloor \log_2(X_i - X_{i-1}) \rfloor$ 
16.    $M_i \leftarrow (X_i - X_{i-1}) \gg k$   $\triangleright$  valor do declive m do formato XMB
17.    $B_i \leftarrow Y_i - ((M_i \cdot X_i) \gg k)$   $\triangleright$  valor da ordenada na origem b do formato XMB
18. end for

```

Figura 1: Pseudocódigo do Algoritmo Greedy para Tabelas Spline XF e XMB

2.2.5 Ferramentas e Tecnologias

O desenvolvimento foi realizado em C++ standard, sem dependências externas além da biblioteca matemática padrão. A biblioteca de emulação LNS (`lnssim.hpp`) e a biblioteca de emulação BF (`bffloatsim.hpp`), ambas desenvolvidas por mim, são cabeçalhos auto-contidos que implementam os respectivos tipos numéricos como templates C++. Para a geração das tabelas de spline, desenvolvi uma ferramenta autónoma (`spline`) que ajusta aproximações por troços às funções f^+ e f^- e escreve os resultados em ficheiros binários em formato `.lns`. Os language models utilizados nos testes de inferência são da família *TinyStories* [9] (`stories15M` e `stories42M`), baseados na arquitectura Llama 2.

3 Descrição do Trabalho

Esta secção apresenta uma descrição detalhada do trabalho desenvolvido no decorrer do estágio, o qual teve como objetivo principal dar continuidade e aprofundar o ecossistema computacional planeado, focando-se em quatro partes fundamentais. Inicialmente, o trabalho centrou-se em adicionar operações de comparação e conversão entre os formatos FP/BF e LNS. De forma a estabelecer uma base prática de exploração do formato, procedeu-se à criação de uma biblioteca de emulação LNS. Subsequentemente, realizou-se uma análise rigorosa para comparar o LNS ao BF em termos de precisão aritmética recorrendo a testes estatísticos. Por fim, este ecossistema foi validado ao suportar inferência em LLMs, focando-se especificamente na arquitetura Llama 2.

3.1 Comparações em LNS e Conversão FP/BF ↔ LNS

3.1.1 Comparações em LNS

Sejam x e y dois operandos, podemos estabelecer as seguintes 5 comparações:

$$x < y, \quad x \leq y, \quad x = y, \quad x \geq y, \quad x > y$$

sendo que apenas é necessária a implementação de duas para suporte da totalidade, via inversão da ordem de operandos, ou combinação de operações:

$$x < y \text{ e } x = y$$

Ou seja, para determinar se x é igual a y , basta comparar os bits um a um, logo, é equivalente a fazer uma comparação de inteiros. Para determinar se x é menor do que y :

$$\text{LNS-LessThan}(x, y) = \begin{cases} \text{true} & \text{se } \text{sign}(x) = 1 \wedge \text{sign}(y) = 0 \\ \text{exponent}(x) > \text{exponent}(y) & \text{se } \text{sign}(x) = \text{sign}(y) = 1 \\ \text{exponent}(x) < \text{exponent}(y) & \text{se } \text{sign}(x) = \text{sign}(y) = 0 \\ \text{false} & \text{caso contrário} \end{cases}$$

Portanto, todas as restantes comparações são estabelecidas através de reformulações equivalentes das inequações:

$$x \leq y \Leftrightarrow x < y \vee x = y \quad x \geq y \Leftrightarrow y \leq x \quad x > y \Leftrightarrow y < x$$

3.1.2 Funções de Conversão FP/BF ↔ LNS

Uma componente importante do trabalho foi a análise formal das funções de conversão entre FP e LNS. Dado que $x_{\text{fp}} = (-1)^{S_x} \cdot (1 + m_x) \cdot 2^{e_x - 127}$ e $x_{\text{lns}} = (-1)^{S_x} \cdot 2^{L_x}$, a igualdade impõe $L_x = e_x - 127 + \log_2(1 + m_x)$, o que define as funções de conversão:

$$\begin{aligned} \text{float2lns}(S_x, e_x, m_x) &:= (-1)^{S_x} \cdot (e_x - 127 + \log_2(1 + m_x)) \\ \text{lns2float}(S_x, L_{x_i}, L_{x_f}) &:= (-1)^{S_x} \cdot \left(1 + \left(2^{L_{x_f}} - 1\right)\right) \cdot 2^{L_{x_i} + 127} \end{aligned}$$

Ambas as expressões não lineares das funções de conversão, $\log_2(1 + m_x)$ e $2^{L_{x_f}} - 1$, têm domínio $[0, 1)$ e podem ser aproximadas com splines lineares. Os upper bounds do erro correspondentes foram derivados analiticamente, seguindo a mesma metodologia aplicada às funções de Gauss logarítmicas:

$$\varepsilon_{\text{float2lns}} \leq \frac{1}{8} \cdot \frac{1}{\ln(2)} \cdot \frac{1}{(1 + x_{i-1})^2} \cdot h_i^2 \quad \varepsilon_{\text{lns2float}} \leq \frac{1}{8} \cdot \ln(2) \cdot 2^{2x_i} \cdot h_i^2$$

3.2 Biblioteca de Emulação LNS

3.2.1 Arquitectura da Biblioteca

A biblioteca de emulação LNS é implementada como um conjunto de cabeçalhos C++ auto-contidos que partilham a mesma interface template `lns<N, I, F>`, parametrizada pela largura total em bits (**N**), pelo número de bits inteiros (**I**) e pelo número de bits fraccionários (**F**). Esta interface expõe os operadores aritméticos C++ standard (+, -, *, /, comparações, etc.) e permite que o mesmo código de aplicação seja compilado para dois backends distintos por simples substituição do `#include`:

- `lnssim.hpp` — emulação em software com tabelas de spline pré-calculadas (formato `.lns`), destinado a simulação e testes no host. A aritmética de adição e subtracção é resolvida por pesquisa na tabela XF ou XMB, dependendo de qual foi lida via `lns{8,16}_read_tables()`. Todas as restantes operações são exactas no domínio logarítmico.
- `lns.hpp` — interface para alvo em hardware que mapeia cada operador directamente para a instrução RISC-V personalizada correspondente via assembly inline, destinado à execução no processador RISC++ em FPGA.

A biblioteca também inclui `bfloatsim.hpp`, um cabeçalho auto-contido que implementa os tipos `bf8` (E4M3) e `bf16` (E7M8) como templates C++. O `bf16` é simulado truncando os 16 bits inferiores de uma representação `fp32` (round-to-zero); o `bf8` aplica a mesma estratégia com a máscara E4M3. Ambos os tipos expõem a mesma interface que os tipos LNS correspondentes, permitindo que o mesmo benchmark seja instanciado sobre qualquer combinação de formatos.

Os tipos concretos utilizados ao longo deste trabalho são definidos como:

```
typedef lns<32, 8, 23> lns32; // acumulador de alta precisão
typedef lns<16, 8, 7> lns16; // formato principal Q8.7
typedef lns<8, 4, 3> lns8; // formato de 8 bits Q4.3
```

`lns32` é utilizado como referência para a comparação de precisão: os erros de `lns8` e `lns16` são calculados relativamente a `lns32`, enquanto os erros de `bf8` e `bf16` são calculados relativamente a `fp32`. Esta metodologia permite comparar cada formato de menor número de bits com um com uma referência de maior resolução dentro da mesma família.

A biblioteca pode ser integrada em qualquer projecto C++ com uma única directiva `#include`, sem dependências externas e sem sistema de build específico. A geração das tabelas usadas pelos formatos de `lns` de 8 e 16 bits é feita por uma custom tool chamada `spline`. Esta gera-as para ficheiros binários `.lns` e permite a customização do diretório em que os escreve. A descrição do formato do ficheiro das tabelas e exemplo de utilização da ferramenta para geração assim como as interfaces mencionadas acima e um exemplo da utilização de uma delas encontram-se no [Anexo](#).

3.3 Comparação de Precisão Aritmética: LNS vs. BF

3.3.1 Metodologia

Foi desenvolvido um benchmark abrangente (`examples/bench/`) que compara a precisão aritmética de LNS e BF nos pares `lns8/bf8` e `lns16/bf16`, cobrindo duas dimensões: precisão por intervalos em quatro operações aritméticas e round-trip (Benchmark 1) e precisão em nove kernels (Benchmark 2).

Benchmark 1 — Operações Aritméticas e Round-Trip

As cinco operações avaliadas são: conversão e retorno (*round-trip*), multiplicação, divisão, adição e subtração, parametrizadas por intervalos `[lo, hi]` (potências de dois consecutivas).

A métrica primária é diferenciada por operação: erro relativo médio (MRE) para round-trip, multiplicação e divisão (invariante à escala); erro absoluto médio (MAE) para adição e subtração, onde o erro relativo diverge em situações de cancelamento. Aplica-se um filtro de cancelamento: pares em que $|a + b| < 2^{-f} \cdot \max(|a|, |b|)$ são descartados, onde f é o número de bits fracionários.

Os vencedores são determinados por teste Mann-Whitney U bilateral com $n = 100,000$ amostras ($p < 0.01$, $|r| \geq 0.05$).

Benchmark 2 — Kernels

Nove testes de nível algorítmico: progressão geométrica, norma euclidiana, série harmónica alternada (Leibniz), acumulação da soma de $\frac{\pi^2}{6}$ com a definição da série do Basel Problem tanto do primeiro até ao termo N e o reverso, sigmoid, GELU, soma de softmax e RMSNorm. LNS é avaliado contra `lns32`; BF contra `f32`. Para cada teste em formato de 16 bits, é testada soma acumulada em 32 bits, `lns16` testado com `lns32` e `fp32`, enquanto que o `bf16` tem soma acumulada apenas com `fp32`. É considerado que há em empate em precisão quando a diferença relativa entre erros é inferior a 5%.

3.4 Inferência na Arquitetura Llama 2

3.4.1 Conversão de Modelos fp32 para lns16 e bf16

Foram desenvolvidas duas ferramentas de conversão de pesos em `examples/tinystories/convert/`: `convert_lns16.cpp` e `convert_bf16.cpp`. Ambas operam em modo de streaming em lotes de 4096 pesos de cada vez, mantendo o cabeçalho de configuração inalterado e convertendo apenas os pesos e os scores do tokenizer.

A conversão $\text{fp32} \rightarrow \text{lns16}$ aplica, para cada peso x_{fp32} :

$$L_x = (e_x - 127) + \log_2(1 + m_x)$$

onde e_x é o expoente IEEE 754 e m_x é a mantissa normalizada em $[0, 1)$. O termo $\log_2(1 + m_x)$ é avaliado via a tabela `float2lns` pré-calculada pelo algoritmo greedy. Esta conversão não requer re-treino. A conversão $\text{fp32} \rightarrow \text{bf16}$ trunca os 16 bits inferiores da representação fp32 , mantendo sinal, 8 bits de expoente e 7 bits de mantissa.

3.4.2 Inferência lns16 vs bf16

O ficheiro `examples/tinystories/tiny/tiny_lns16.cpp` implementa o ciclo completo de inferência do transformer Llama 2 usando exclusivamente `lns16` para pesos e activações. As operações fundamentais foram re-implementadas com o tipo `lns16`: multiplicação de matrizes (`matmul`), atenção multi-cabeça, normalização RMS, SwiGLU e softmax.

A acumulação extensa é particularmente sensível ao erro de aproximação da adição LNS, dado que este se acumula a cada termo somado. Esta sensibilidade afecta todas as operações que somam ao longo de muitos termos: a soma dos quadrados na normalização RMS, o produto escalar e a soma ponderada da atenção, a soma do denominador do softmax, as multiplicações de matrizes (`matmul`) e o *residual stream* entre camadas. Adoptou-se por isso, em todas estas operações, uma abordagem híbrida em que a soma é calculada num acumulador de precisão alargada e só o resultado final é convertido para `lns16`; a multiplicação escalar entre termos individuais permanece sempre em `lns16` puro.

No ficheiro `tiny_lns16.cpp`, o acumulador de precisão alargada utilizado é `fp32`. Um processo equivalente foi implementado no ficheiro `tiny_lns16_lns32acc.cpp`, em que se substitui `fp32` por uma simulação de `lns32` (*drop-in replacement*) nas mesmas operações. Foi também testada uma versão equivalente em `bf16` (ficheiro `tiny_bf16.cpp`), com `fp32` como acumulador de precisão alargada.

4 Resultados e Análise

Esta secção apresenta e discute os resultados obtidos nos testes experimentais, estruturando-se em dois eixos principais de avaliação. Primeiro, expõe-se uma análise comparativa detalhada da precisão aritmética entre os formatos LNS e BF. Segundo, demonstra-se a viabilidade prática destes formatos através da sua aplicação direta na inferência de language models da arquitetura Llama 2.

4.1 Comparação de Precisão Aritmética: LNS vs. BF

4.1.1 Benchmark 1 — Operações Aritméticas e Round-Trip

Arithmetic Accuracy: MRE & MAE for 8-bit & 16-bit formats

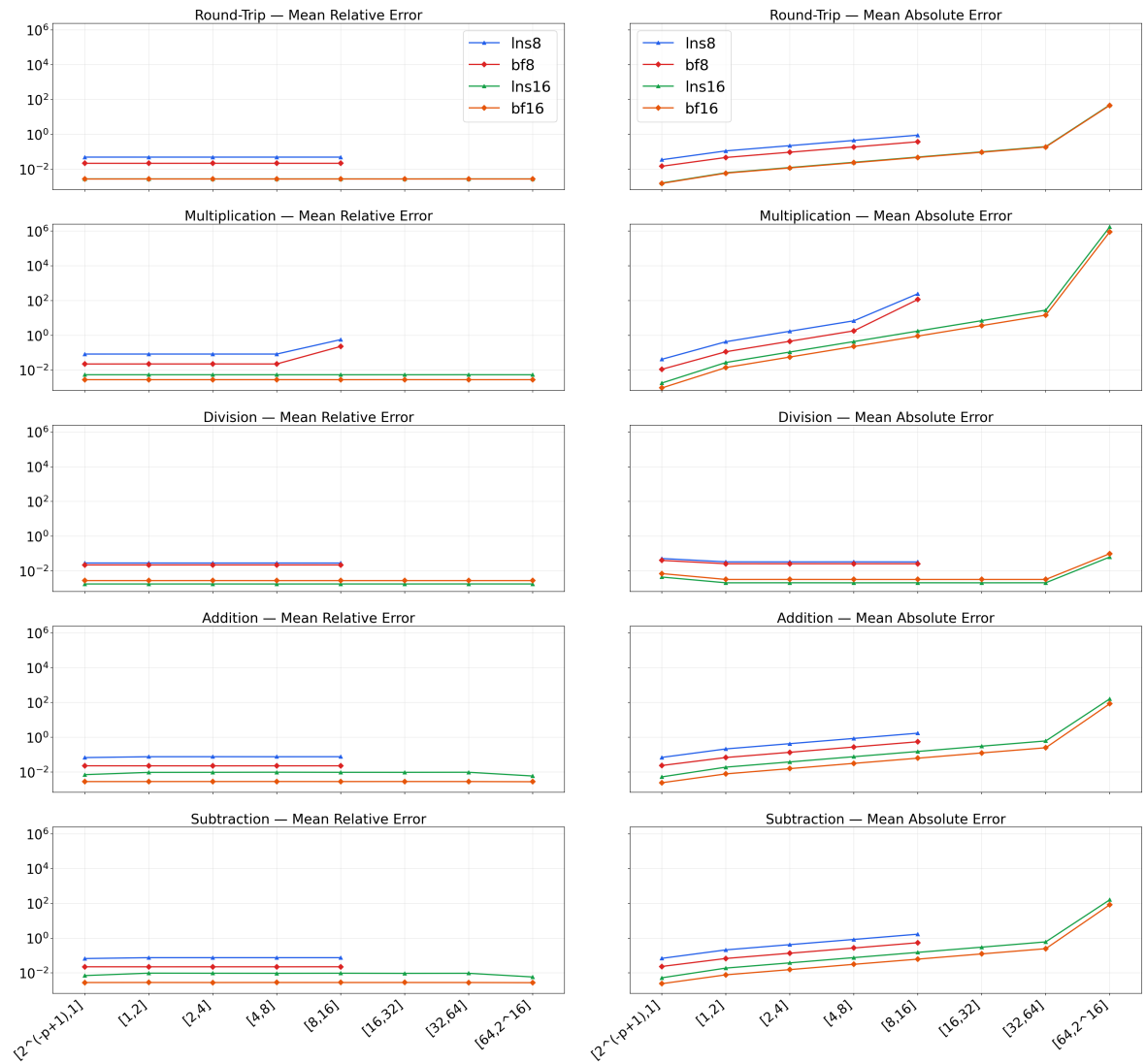


Figura 2: Erro relativo e absoluto médio por intervalo de valores para os operadores. (p é o número de bits da parte inteira do expoente em LNS — 2^{-p+1} é o número mais pequeno que ambos os formatos conseguem representar)

O erro relativo é aproximadamente invariante ao intervalo para todos os formatos; a diferença LNS/BF é essencialmente constante ao longo dos intervalos.

O erro absoluto cresce com a magnitude para todos os formatos. Os gráficos de adição, subtracção, multiplicação e round-trip mostram que o erro absoluto de ambos os formatos é crescente para maiores magnitudes de operadores, consistente com a acumulação de erro na correcção de adição em log-space.

Heatmap — Winner per Operation × Interval (MRE | MAE)

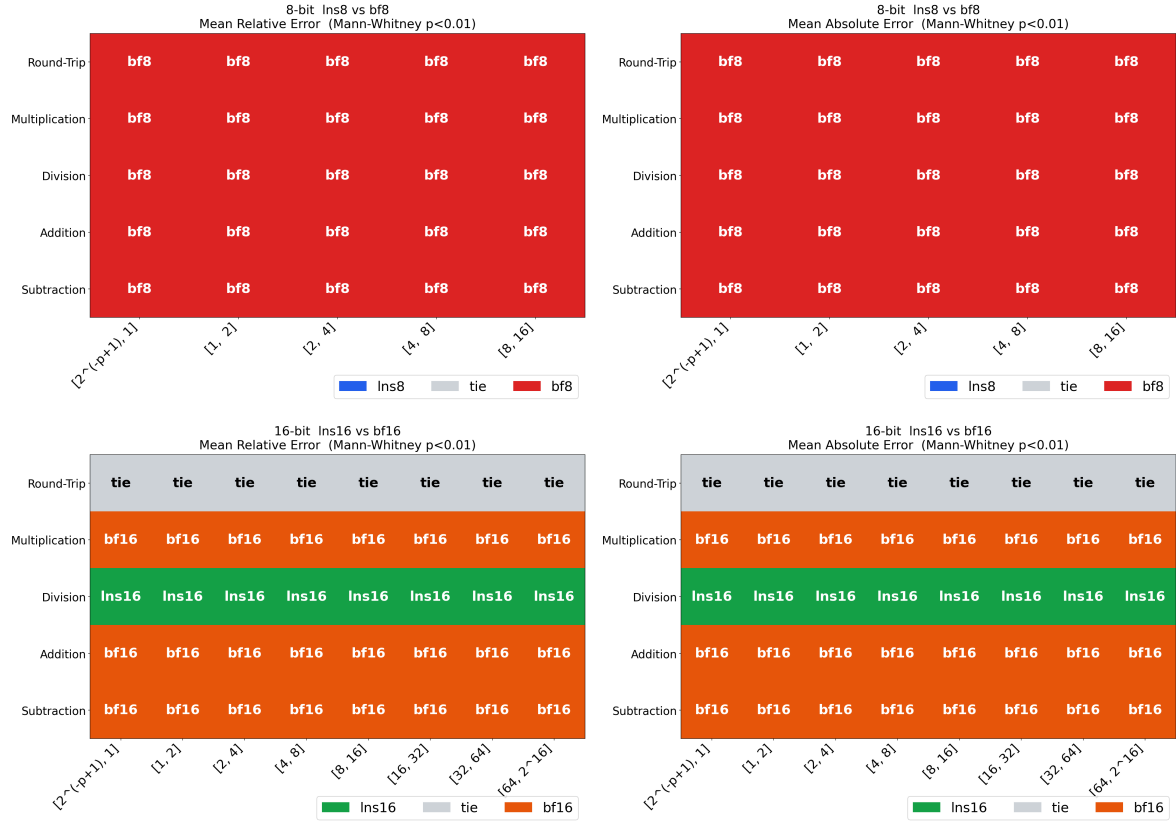


Figura 3: Comparação de heatmaps de vencedor por operação × intervalo para os formatos de 8 bits (lns8 vs bf8). À esquerda: erro relativo médio; À direita: erro absoluto médio.

Em erro relativo e absoluto, o bf8 vence em todos os três intervalos sem empates. O resultado de multiplicação e divisão é digno de nota: embora a aritmética LNS seja exata no expoente e o bf8 sofra com apenas 3 bits de mantissa, este permanece mais preciso. Adicionalmente, o resultado de adição e subtracção confirma a fraqueza conhecida do LNS, onde o termo de correcção aproximado com a tabela introduz um erro que a adição directa do bf8 não sofre.

Em erro absoluto, o lns16 vence a multiplicação em 7 dos 8 intervalos e a divisão em todos os 8; o bf16 vence adição e subtracção em todos os intervalos. Em erro relativo, o bf16 inverte o resultado na multiplicação. Esta inversão é significativa: o lns16 é preferível quando os operandos permanecem numa gama limitada e a fidelidade absoluta importa; o bf16 é preferível quando a precisão relativa tem de ser mantida num intervalo dinâmico amplo.

É preciso também realçar que estes resultados são explicitamente para os formatos declarados na Secção 2.2.2 e que outros formatos de lns8 e lns16 poderão ter melhores resultados com os análogos em BF.

4.1.2 Benchmark 2 — Kernels

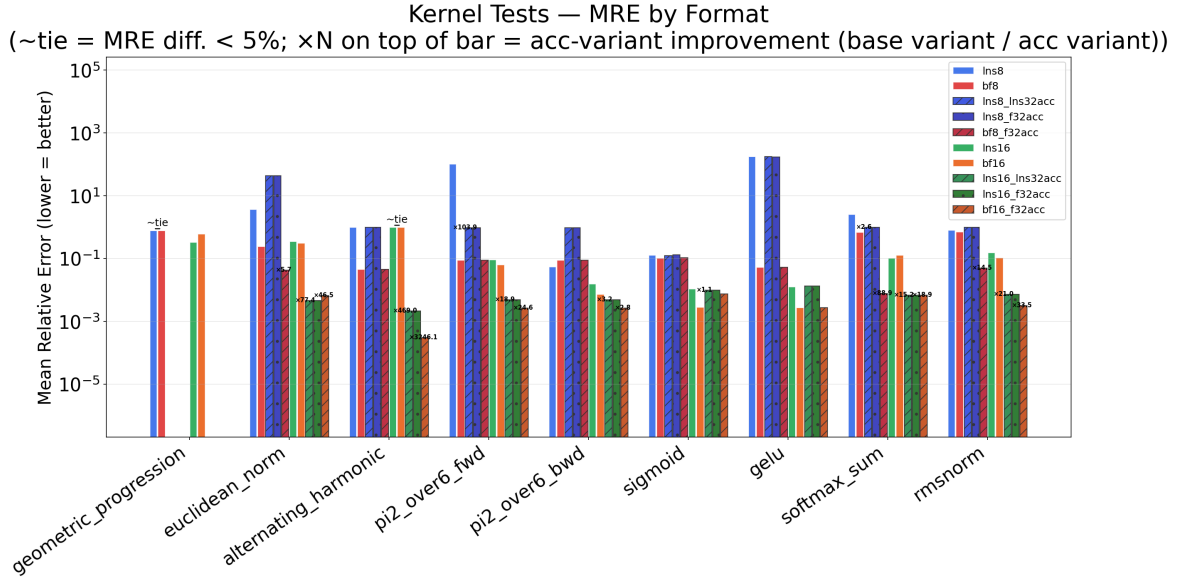


Figura 4: Erro relativo médio dos testes numéricos para todos os formatos. Os empates são anotados quando a diferença relativa entre lns e bf é inferior a 5%.

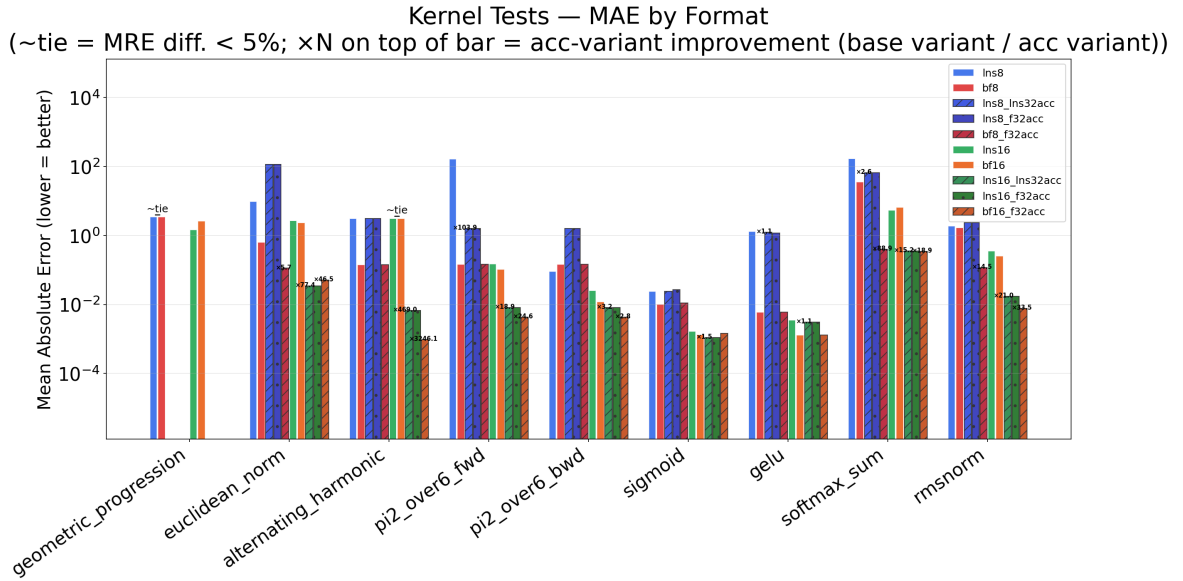


Figura 5: Erro absoluto médio dos testes numéricos para todos os formatos. Os empates são anotados quando a diferença relativa entre lns e bf é inferior a 5%.

A Figura 4 e a Figura 5 mostram que os resultados dos kernels são consistentes com a avaliação das operações por intervalo: em qualquer workload dominado pela subtração ou adição, a adição corretamente arredondada do **bf16** preserva maior precisão. As variantes de acumulação em 32 bits mostram que esta diferença é praticamente devida à precisão na acumulação em vez do formato: `lns16_lns32_acc` e `lns16_f32acc` apresentam melhor precisão em testes como `pi2_over6`, `softmax_sum` e `rmsnorm_denom` quando comparados com os casos análogos sem acumulação de maior precisão. Em variantes de puras sequências de multiplicação as variantes de 16 bits em LNS têm melhor precisão do que BF. Nos formatos de 8 bits, à uma maior competitividade, o `lns8` mostra-se por vezes melhor que o `bf8` ou equivalente. É de realçar também, que em `lns8`, as versões de acumulação em 32 bits nem sempre melhoram o erro devido ao erro constante de conversão entre formatos.

4.2 Inferência na Arquitetura Llama 2

Todos os modelos foram corridos com os parâmetros default exceto o tokenizer que muda de acordo com o formato LNS/BF.

Parâmetro	Descrição	Valor Default
Temperature	Controla criatividade; 0.0 é determinístico, 1.0 é criativo	1.0
Top-p	Nucleus sampling; remove 1 – topp dos tokens menor prováveis	1.0
Steps	Máximo número de tokens da geração	256
Prompt	Texto inicial que o modelo usa como baseline para começar	NULL
rng_seed	Semente do gerador de números aleatórios; 0 usa system time	0
Mode	Determina como o modelo se comporta (generate/chat)	generate
System Prompt	Usado apenas em modo chat para maior instruction context	NULL

Tabela 2: Parâmetros de inferência utilizados nos modelos Llama 2.

De seguida, apresentam-se exemplos de texto gerado por cada variante testada (l_{ns16} XF, l_{ns16} XMB e b_{f16}), para os dois modelos utilizados, stories15M e stories42M.

Stories 15M

- l_{ns16} (XF)

Once upon a time, there was a little girl named Lily. She loved to draw and color with her crayons. One day, she drew a beautiful picture of a rainbow with lots of colors. Her mom saw the picture and said, "Lily, that's a pretty flower. You should try to make it even more beautiful." Lily smiled and said, "I want to design a new picture of a rainbow!" Her mom gave her some paper and crayons and Lily started to draw. But then she got confused and drew the same big, bright colors. She said, "Mommy, this picture is not perfect." Her mom said, "Oh no, I'm sorry. We can't print it. But let's draw another one." Lily learned that sometimes things don't go the way we want them to, but it's important to keep trying and not give up.

- l_{ns16} (XMB)

One day, a boy named Tim and a girl named Sue went to the beach. They saw a lot of water and a big, pretty sun. They wanted to play and splash in the water. Tim had a toy boat that he would push in the water. Sue had a ball that she liked to throw. Tim said, "Let's race our boats!" Sue agreed and they put their boats in the water. Tim's boat went very fast, and Sue's ball went in the water. They laughed and clapped their hands as they watched the boats swim. After a while, Tim felt tired. He said, "I'm hungry. Let's stop and eat." Sue agreed and they started to eat their lunch. They sat on the sand and watched the boats in the water. They had a fun day at the beach and went home happy.

- b_{f16}

Once upon a time, there was a little girl named Lily. She had a big brother named Max who was very dependable. Max always helped Lily when she needed it. One day, Lily wanted to play with her toys but Max said no. Lily was sad and angry. She wanted to play with Max's toys. Lily's mom noticed her daughter was upset and asked her what was wrong. Lily explained that Max wouldn't let her play with his toys. Lily's mom talked to Max and told him that Lily was being mean. Max didn't listen and kept saying mean things. Lily felt better and decided to talk to Max. She told him that she didn't want to play with him and that it was okay to be mean. Max realized he was being mean and gave Lily a toy to play with. They both played together and had fun.

Stories 42M

- lns16 (XF)

Once upon a time, there was a little boy named Tim. Tim was very excited because he was going on a holiday with his family. They were going to the beach! Tim loved the beach because he could play in the sand and swim in the sea. One day, Tim's family decided to have a picnic in the park. They packed sandwiches and fruit to eat. But when they went to the beach, they saw that the sand was not there! They looked everywhere but couldn't find it. Tim was sad because he really wanted to eat the sandwich. Suddenly, they heard a loud noise and saw a big wave coming towards them. They were scared because it was a storm. But then, the wave brought a big box with them. They opened the box and found a treasure! It was a big toy boat that Tim and his family could use. They were so happy and thanked the wave for bringing the treasure.

- lns16 (XMB)

Once upon a time, there was a little girl named Lily. She loved to help her mom in the kitchen. One day, her mom asked her to get some ice from the freezer. Lily was so happy to help! She ran to the freezer and opened it. Inside, she found a rare ice cream. Lily was so excited to eat it, but then she saw something she didn't expect. The ice cream started to melt in her hands! She quickly took it off and ran to her mom. "Mommy, the ice cream is melting! It's making me panic!" she exclaimed. Her mom laughed and said, "Don't worry, Lily. It's just a melted ice cream. Let's go back to the kitchen and get more." Lily was happy to get more ice cream, but she knew she had to wait for it to melt away.

- bf16

Once upon a time, there was a little boy named Tim. Tim was a pupil in a small school. He had a friend named Sam. Sam was a very reliable friend. They liked to play and laugh together. One day, Tim and Sam went to play outside. They saw a big tree and wanted to climb it. Tim said, "Let's go up, Sam!" Sam agreed, and they started to climb the tree. As they climbed, they saw a bird in the tree. The bird said, "Hello, Tim and Sam!" Tim and Sam were very surprised. They did not know that birds could talk. The bird said, "I am a magic bird. If you help me, I will give you a surprise." Tim and Sam thought about it and said, "Yes, we can help you!" The magic bird told Tim and Sam to close their eyes. When they opened them, they were now in a magical world. They met many animals and had many fun adventures. They were very happy and thankful for the magic bird and his help.

4.2.1 Análise

A produção de texto coerente pelos modelos *TinyStories* após conversão directa $\text{fp32} \rightarrow \text{lns16}$, sem qualquer re-treino, confirma a viabilidade prática do lns16 Q8.7 para inferência em redes neuronais. Esta coerência mantém-se tanto na variante XF como na XMB, e em ambos os modelos testados (stories15M e stories42M), o que sugere que a precisão de 16 bits oferecida pelo LNS é suficiente para preservar a estrutura gramatical e narrativa do texto gerado, independentemente da representação específica adoptada para as funções de correcção. O modelo bf16, usado aqui como termo de comparação, exhibe um comportamento semelhante, produzindo texto igualmente fluente e gramaticalmente correcto, sem degradação perceptível em relação ao modelo de referência em fp32.

Importa notar que estas observações são qualitativas e baseadas numa única amostra por configuração; não permitem, por si só, quantificar diferenças de qualidade entre LNS e BF, nem garantir a ausência de degradação num conjunto mais amplo de prompts.

5 Conclusão e Trabalho Futuro

Este estágio aprofundou e alargou uma implementação LNS prévia em quatro direções: operações de comparação e conversão, biblioteca de emulação, comparação estatística com BF e inferência em language models. Os resultados permitem tirar conclusões mais matizadas do que uma simples superioridade de um formato sobre o outro, identificando antes os regimes e workloads em que cada um se torna preferível.

No regime de 8 bits, o **bf8** venceu o **lns8** em todas as operações testadas no Benchmark 1, incluindo multiplicação e divisão — apesar da exactidão estrutural do LNS no domínio logarítmico, a sua reduzida largura de mantissa penaliza-o de forma mais severa do que o **bf8** consegue absorver com os seus 3 bits de mantissa. Esta vantagem do **bf8**, no entanto, não se reproduziu de forma tão consistente nos kernels do Benchmark 2, onde o **lns8** por vezes superou o **bf8**, embora a acumulação em 32 bits nem sempre tenha melhorado o erro do **lns8**, devido ao erro constante introduzido na conversão entre formatos. No regime de 16 bits, a assimetria entre erro absoluto e relativo torna-se o resultado mais relevante: o **lns16** mantém vantagem em erro absoluto na multiplicação (7 de 8 intervalos) e na divisão (8 de 8), mas o **bf16** inverte este resultado em erro relativo na multiplicação e vence a adição e subtracção em todos os intervalos testados. Esta inversão sugere uma recomendação de uso concreta: o **lns16** é preferível quando os operandos permanecem numa gama limitada e a fidelidade absoluta importa, enquanto o **bf16** é preferível quando é necessário manter precisão relativa num intervalo dinâmico amplo. Os testes em funções não-lineares compostas reforçam que esta vantagem não é universal: o **lns16** apresenta maior erro do que o **bf16** na GELU, mas menor erro na Softmax, mostrando que o desempenho do LNS depende também da estrutura específica de cada função e não apenas da operação aritmética subjacente.

A descoberta mais relevante para a viabilidade prática do formato é o efeito da acumulação em precisão alargada: testes com acumuladores em **lns32** e **fp32** mostram que a fraqueza do LNS em workloads de acumulação intensiva — como a soma de $\frac{\pi^2}{6}$, a soma do denominador do softmax e o denominador do RMSNorm — é significativamente atenuada quando a soma é calculada num acumulador de maior precisão. Esta mitigação foi a base da abordagem híbrida adoptada na inferência: a multiplicação escalar entre termos individuais permanece em **lns16** puro, exacta por construção, enquanto a sua soma é acumulada em **lns32** ou **fp32**. Com esta abordagem, a conversão directa de pesos **fp32** para **lns16**, sem qualquer re-treino, produz texto coerente nos modelos stories15M e stories42M — tanto na representação XF como na XMB —, confirmando que a combinação de multiplicação exacta com acumulação de precisão alargada é suficiente para preservar a qualidade da inferência, mesmo num cenário de conversão directa sem fine tuning.

Como **trabalho futuro**, os eixos mais imediatos são: investigar a causa do erro constante de conversão que limita os ganhos da acumulação em precisão alargada no **lns8**, de forma a tornar esta mitigação também eficaz em formatos de menor largura; implementar métodos de aproximação de maior ordem — diferenças divididas de Newton e Chebyshev por troços — para reduzir o número de intervalos necessários em f^- e tornar praticáveis os formatos **lns32** e **lns64**; avaliar o LNS em arquitecturas convolucionais e em language models de maior escala; e explorar re-treino ou fine tuning com representação LNS para recuperar a precisão perdida na conversão directa, potencialmente reduzindo a dependência da abordagem híbrida nas operações de acumulação.

Bibliografia

- [1] H. Teixeira e J. Paradela, «AI Accelerator for a Core RISC-V», technical report, 2026. doi: [10.5281/zenodo.21010201](https://doi.org/10.5281/zenodo.21010201).
- [2] B. Parhami, «Logarithmic Number Systems: Theory and Applications», *Advances in Computers*, vol. 117, p. 106800, 2020.
- [3] J. N. Coleman, E. I. Chester, C. I. Softley, e J. Kadlec, «Lookup-Tables for Logarithmic Number System Operations», *IEEE Transactions on Computers*, vol. 49, n.º 7, pp. 702–715, 2000.
- [4] M. Christ e others, «Low-Precision Logarithmic Arithmetic for Neural Network Accelerators», em *Proceedings of the IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2022.
- [5] Y. Wu e others, «PVU: A Posit Vector Processor Unit Based on RISC-V Extension». 2025.
- [6] B. D. Rouhani e others, «Microscaling Data Formats for Deep Learning». 2023.
- [7] E. Islamoglu e others, «MXDOTP: A RISC-V ISA Extension for Enabling Microscaling (MX) Floating-Point Dot Products». 2025.
- [8] S. Han, H. Mao, e W. J. Dally, «Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding». 2015.
- [9] A. Karpathy, «llama2.c. Available: <https://github.com/karpathy/llama2.c>». 2023.

A Anexo

A.1 APIs

```
#include <lns>

template<u8 n>
struct lns {
    static constexpr u8 f_width =
        n == 8 ? 3 :
        n == 16 ? 7 :
        n == 32 ? 19 : 47;

    f32 bits;

    lns();
    lns(u32 raw);

    explicit operator f32() const;

    lns operator+ (const lns other) const;
    lns operator- (const lns other) const;
    lns operator- () const;
    lns operator* (const lns other) const;
    lns operator/ (const lns other) const;

    lns sqrt () const;

    lns& operator+=(const lns other);
    lns& operator-=(const lns other);
    lns& operator*=(const lns other);
    lns& operator/=(const lns other);

    void operator= (const lns other) volatile;

    bool operator==(const lns other) const;
    bool operator!=(const lns other) const;
    bool operator< (const lns other) const;
    bool operator<=(const lns other) const;
    bool operator> (const lns other) const;
    bool operator>=(const lns other) const;

    static inline lns load (const void* addr, u32 imm = 0);
    inline void store(void* addr, u32 imm = 0) const;
};

using lns8 = lns<8>;
using lns16 = lns<16>;
using lns32 = lns<32>;
using lns64 = lns<64>;
```

```

#include <lnssim>
template<u8 n, u8 i, u8 f>
struct lns {
    uint_t<n> bits;

    lns();
    lns(uint_t<n> raw, bool);
    lns(i32 x);
    lns(f32 x);
    template<u8 n2, u8 i2, u8 f2>
    lns(const lns<n2, i2, f2>& other);

    explicit operator f32() const;
    explicit operator f64() const;

    u8      sign()      const;
    int_t<n> exponent() const;
    bool    is_zero()   const;

    lns operator+ ( const lns other) const;
    lns operator- ( const lns other) const;
    lns operator- () const;
    lns operator* ( const lns other) const;
    lns operator/ ( const lns other) const;

    lns power2_pow ( const u8 k) const;
    lns power2_root ( const u8 k) const;

    lns square      () const;
    lns sqrt        () const;
    lns exp          () const;
    lns sinh        () const;
    lns cosh        () const;
    lns tanh        () const;

    // ... (restantes operadores de atribuição e comparação)
};

```



```

#include <bfloatsim>

struct bf8 {
    u8 bits; // E4M3: [S | EEEE | MMM]

    bf8();
    bf8(u8 raw, bool);
    bf8(i32 x);
    bf8(bf16 x);
    bf8(f32 x);

    operator f32() const;

    inline bool operator== (const bf8 other) const;
    inline bool operator< (const bf8 other) const;
    inline bool operator> (const bf8 other) const;
    inline bool operator<= (const bf8 other) const;
    inline bool operator>= (const bf8 other) const;

    bf8 operator+      (const bf8 other) const;
    bf8 operator-      (const bf8 other) const;
    bf8 operator-      () const;
    bf8 operator*      (const bf8 other) const;
    bf8 operator/      (const bf8 other) const;

    // ... (restantes operadores de atribuição e comparação)
};

struct bf16 {
    ul6 bits; // E8M7: [S | EEEEEEEE | MMMMMMMM]

    bf16();
    bf16(ul6 raw, bool);
    bf16(i32 x);
    bf16(bf8 x);
    bf16(f32 x);

    inline operator f32() const;

    inline bool operator== (const bf16 other) const;
    inline bool operator< (const bf16 other) const;
    inline bool operator> (const bf16 other) const;
    inline bool operator<= (const bf16 other) const;
    inline bool operator>= (const bf16 other) const;

    bf16 operator+      (const bf16 other) const;
    bf16 operator-      (const bf16 other) const;
    bf16 operator-      () const;
    bf16 operator*      (const bf16 other) const;
    bf16 operator/      (const bf16 other) const;

    // ... (restantes operadores de atribuição e comparação)
};

```

A.2 Spline

Os ficheiros `.lns` guardam os dados de interpolação em formato binário bruto (*raw binary*) em modo *Little-Endian*, prontos a serem mapeados diretamente para a memória do simulador.

Cabeçalho Fixo (16 bytes)

O ficheiro começa sempre com quatro inteiros de 32 bits sem sinal (u32) que definem o número de linhas (nós) de cada sub-tabela:

Bytes 0 a 3	Bytes 4 a 7	Bytes 8 a 11	Bytes 12 a 15
s_plus (u32)	s_minus (u32)	s_f2l (u32)	s_l2f (u32)
Tam. Adição	Tam. Subtração	Tam. FP → LNS	Tam. LNS → FP

Layout do Payload (Dados Sequenciais)

Após o cabeçalho, os dados das tabelas surgem de forma contígua na seguinte ordem:

`spline_plus` → `spline_minus` → `spline_f2l` → `spline_l2f`

O formato e o tamanho de cada registo dentro das tabelas dependem da precisão (LNS16/LNS8) e do modo de cálculo (XF/XMB):

Modo XF (Interpolação por Coordenadas): **2/4 bytes por linha:**

x (i8/i16)	y (i8/i16)
------------	------------

Modo XMB (Declive e Translação): **3/6 bytes por linha:**

x (i8/i16)	m (i8/i16)	b (i8/i16)
------------	------------	------------

Para o formato de 8 bits, os dados sofrem um *bit-shift* à direita antes de serem gravados, reduzindo o consumo de memória para metade.

Nas tabelas f2l e l2f do formato LNS8, a operação binária aplicada aos dados originais de 16-bit antes da escrita é: `byte = (i16_val >> 8) & 0xFF`.

Geração e Customização de Tabelas (spline)

A ferramenta CLI em `lib/spline/` gera e testa as tabelas binárias `.lns`. O binário suporta dois modos de operação via *flags* estritas:

```
# Compilação e geração automática dos perfis padrão (Q8.7 e Q4.3)
cd lib/spline && make
```

Sintaxe da CLI

```
bash ./build/spline <--gen | --test> [config]
```

- `--gen`: Exporta os ficheiros `.lns`.
 - requer 4 valores: `[+, -, f2l, l2f]` (linhas: `[2, 1024]`).
- `--test`: Print do erro de aproximação no `stdout`.
 - requer 1 valor: `[max_linhas]` (`[2, 1024]`).
- `[config]`
 - `--xf / --xmb`: pontos vs. declive (m, b).
 - `--lns16 / --lns8`: largura do formato.
 - `<int_digits>`: bits inteiros; intervalo de valores possíveis em `lns8` é `[4, 6]` e em `lns16` é `[4, 14]`.

Exemplos Práticos

A. Geração Personalizada (--gen)

Cria tabelas em modo **XMB** para **lns16** (Q8.7) com tamanhos assimétricos por sub-tabela:

```
# Ordem: + - f2l l2f | int: 8
./build/spline --gen --xmb 128 256 64 64 --lns16 8
```

B. Teste de Precisão (--test)

Avalia o impacto do número de linhas no erro para **lns8** (Q4.3) em modo **XF** (sem gravar ficheiros):

```
# Varrimento até 128 linhas | int: 4
./build/spline --test --xf 128 --lns8 4
```

Nota: O modo `--test` mapeia o par Erro vs Linhas para validação rápida antes da síntese em FPGA.

Exemplo de Utilização das Tabelas

```
#define SPLINE_XMB
#include <lnssim.hpp>

using lns8 = lns<8, 4, 3>;
using lns16 = lns<16, 8, 7>;

lns8_read_tables ("lib/spline/lns_tables/lns8_q4_3_xmb.lns");
lns16_read_tables("lib/spline/lns_tables/lns16_q8_7_xmb.lns");

lns16 a(1.5f), b(2.0f);
float result = (float)(a * b); // exact – integer add on exponents
float result2 = (float)(a + b); // approximated via spline LUT
float result3 = (float)a.sqrt(); // exact – exp >= 1

lns_close();
```